

Core Properties of Cryptographic Hash Functions

Objective

After completing this topic, students will be able to:

- Explain the **three security properties** of hash functions.
 - Understand why these properties are essential for **data integrity and authentication**.
 - Illustrate these ideas using **Python examples** and real-world analogies.
-

1. Introduction to Cryptographic Hash Functions

A **hash function** takes input data (a message of any length) and produces a **fixed-size digest** (a sequence of bits).

For example:

```
H("Hello World") = a591a6d40bf4204a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
```

This is the **SHA-256 hash** of the string `"Hello World"` — always 256 bits long.

In **cryptography**, we expect certain *security properties* from hash functions to ensure data safety.

2. The Three Fundamental Security Properties

These are the pillars of any *secure hash function*:

Property	Mathematical Expression	Goal
Preimage Resistance	Given h , it's infeasible to find x such that $H(x) = h$	Can't reverse the hash
Second Preimage Resistance	Given x_1 , it's infeasible to find another $x_2 \neq x_1$ such that $H(x_1) = H(x_2)$	Can't find another input with same hash
Collision Resistance	It's infeasible to find any two inputs $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$	Can't find <i>any</i> pair of collisions

3. Detailed Explanation and Examples

A. Preimage Resistance (One-wayness)

Definition:

Given a hash output h , it should be *computationally impossible* to find any input x such that:

$$H(x) = h$$

Meaning:

Even if you know the hash value, you cannot “reverse-engineer” it to get the original message.

Example:

```
import hashlib

# Suppose attacker only knows this hash
h = hashlib.sha256(b"HelloWorld").hexdigest()
print("Known hash:", h)
```

Output:

Known hash: 872e4bdc3f390d2f99b6d914f2a3c88b1f58f0d106ce8b1c25c78c709d9a52b7

To find the original input, an attacker must try random guesses — but since SHA-256 has (2^{256}) possibilities, **brute-forcing is unrealistic**.

B. Second Preimage Resistance

Definition:

Given one message x_1 , it should be infeasible to find another message $x_2 \neq x_1$ such that:

$$H(x_1) = H(x_2)$$

Meaning:

If someone gives you a message and its hash, you should not be able to find a *different* message that gives the same hash.

Example:

```
msg1 = b"Assignment Marks: 80"
msg2 = b"Assignment Marks: 81"

h1 = hashlib.sha256(msg1).hexdigest()
h2 = hashlib.sha256(msg2).hexdigest()

print("H(msg1):", h1)
print("H(msg2):", h2)
print("Hashes Match?", h1 == h2)
```

Output:

```
Hashes Match? False
```

Even a tiny change (80 → 81) produces a completely different hash — this is called the **avalanche effect**.

C. Collision Resistance

Definition:

It should be infeasible to find *any two distinct messages* x_1 and x_2 such that:

$$H(x_1) = H(x_2)$$

Meaning:

No two different inputs should produce the same hash.

Example:

For secure hashes like SHA-256, collisions are so rare that they haven't been found. But for **MD5** and **SHA-1**, collisions exist.

Let's test with MD5:

```
import hashlib

msg1 = b"Hello World"
msg2 = b"hello world"

h1 = hashlib.md5(msg1).hexdigest()
h2 = hashlib.md5(msg2).hexdigest()

print("MD5(msg1):", h1)
print("MD5(msg2):", h2)
print("Hashes Match?", h1 == h2)
```

Output:

```
MD5(msg1): b10a8db164e0754105b7a99be72e3fe5
MD5(msg2): 3e25960a79dbc69b674cd4ec67a72c62
Hashes Match? False
```

So here, no collision — but researchers *can* find two different files (like PDFs) with the **same MD5 hash**, proving MD5 is not collision-resistant.

4. Real-World Analogy

Concept	Analogy
Hash Function	A blender: whatever you put in becomes unrecognizable juice.
Preimage Resistance	Given the juice, you can't recreate the original fruit.
Second Preimage Resistance	You can't find a different fruit mix that makes identical juice.
Collision Resistance	No two different fruit mixes should ever produce exactly the same juice.

5. Summary / Key Takeaways

Property	What It Prevents	Practical Importance
Preimage Resistance	Reverse-engineering message from hash	Protects stored passwords
Second Preimage Resistance	Modifying a message without changing its hash	Prevents forgery of signed messages
Collision Resistance	Creating two different messages with same hash	Ensures trust in digital signatures & blockchain integrity

Birthday Attack

1) Intuition (plain English)

- A birthday attack exploits the same math as the well-known “birthday paradox”: **you need far fewer than N tries to get a collision in a space of size N .**

- For a hash function with an **n-bit** output there are ($N = 2^n$) possible hashes. The birthday attack finds **any two different messages** ($x_1 \neq x_2$) such that ($H(x_1)=H(x_2)$).
 - Because collisions are *any* pair, not a preimage to a *specific* hash, the work required is about ($2^{\{n/2\}}$) rather than (2^n). That is the key weakness: collision search complexity is the square root of the full space.
-

2) How a birthday attack is used against a hash (practical steps)

- **Goal:** find *any* two different messages (m_1) and (m_2) with the same hash.
- **Approach:**
 1. Generate many candidate messages (often random or chosen with small variations).
 2. Compute hash for each candidate.
 3. Store hashes in a table; when you see the same hash twice you found a collision.
- **Complexity:** about ($O(2^{\{n/2\}})$) hash computations and ($O(2^{\{n/2\}})$) memory (can be time–memory traded).
- **Optimization:** use meet-in-the-middle, distinguished points, or rainbow tables to reduce memory or parallelize the search.

Real-world attack scenario:

- If signature systems sign the **hash** of a file, an attacker can:
 1. Create many “benign” documents until two share the same hash (or craft one benign and one malicious pairing).
 2. Get the signer to sign the benign document’s hash.
 3. Swap in the malicious document with the same hash — signature verifies although content differs.
 - This is why **collision resistance** is essential where signatures are used.
-

3) Short Python demo — *truncated hash* birthday attack

Purpose: demonstrate concept cheaply by truncating hash output (e.g., to 24 bits). With a small output, collisions happen quickly and students can observe them.

```
# Demo: find a collision on truncated SHA-256 (e.g., 24-bit)
# WARNING: this is educational. Do not use truncated hashes for security.

import hashlib, random, string

def rand_msg(length=8):
    return ''.join(random.choice(string.ascii_letters+string.digits) for _ in
range(length))

def trunc_hash_hex(msg, bits=24):
    h = hashlib.sha256(msg.encode()).hexdigest()
    hex_chars = bits // 4          # 4 bits per hex char
    return h[:hex_chars]

def birthday_demo(bits=24, max_iters=2000000):
    seen = {}
    for i in range(max_iters):
        m = rand_msg(10)
        th = trunc_hash_hex(m, bits)
        if th in seen:
            return (seen[th], m, th, i+1)    # collision found: (msg1, msg2,
truncated_hash, tries)
        seen[th] = m
    return None

# Run
result = birthday_demo(bits=24)
if result:
    m1, m2, h, tries = result
    print("Collision found after", tries, "tries")
    print("msg1:", m1)
    print("msg2:", m2)
    print("truncated hash (24-bit):", h)
else:
    print("No collision found within limit")
```

- For 24 bits, expected collisions around ($\sim 2^{12} = 4096$) tries (since $(2^{24/2}=2^{12})$). Students will see collisions quickly.

- If you increase truncation (e.g., 40 bits), it will take longer — demonstrating the $(2^{n/2})$ scaling.
-

4) Why birthday attack matters (implications)

- **Collision attacks** are realistic against **short** or **broken** hashes (e.g., MD5 and SHA-1 have demonstrated collisions).
 - Where collisions are catastrophic (digital signatures, certificate authorities), an attacker can exploit collision generation to forge signatures or certificates.
 - Systems that rely only on hash equality for security (like deduplication, content signing without extra binding) are vulnerable.
-

5) Defenses & best practices

- **Use sufficiently long hashes:** at least 256 bits (SHA-256) or stronger. Collision work becomes infeasible ($\approx (2^{128})$ ops for SHA-256).
 - **Avoid MD5/SHA-1** for anything requiring collision resistance (signatures, certificates).
 - **Use HMAC** (keyed hash) when you require message authentication — HMAC resists certain structural attacks because attacker lacks key.
 - **Salt and domain separation:** add unpredictable or domain-specific inputs to hashes where appropriate (e.g., password hashing, PBKDF2).
 - **Digital signatures:** sign structured data (not raw file hash blindly) and include metadata (timestamp, identity) so swapping content is harder.
 - **Use memory/time-hard password hashing** (PBKDF2, bcrypt, scrypt, Argon2) for passwords — different problem but related: make brute force more expensive.
-

Exploring Hash Stretching

1 What is Hash Stretching?

◇ Definition:

Hash stretching is a technique used to make a password hashing process slower and more resistant to brute-force attacks by applying a hash function multiple times.

In simple words:

🔄 Instead of hashing once, we hash the result **again and again** (thousands or millions of times).

This doesn't make the password stronger — but it makes **password cracking much slower** for attackers.

2 Why Do We Need Hash Stretching?

Let's recall a problem:

When users store passwords, they are **hashed** and saved, not stored in plain text.

Example:

```
import hashlib
print(hashlib.sha256(b"mypassword").hexdigest())
```

Output:

```
89e01536ac207279409d4de1e5253e01f4a1769e696db0d6062ca9b8f56767c8
```

If an attacker steals this hash, they can:

- Use a **dictionary attack** or **brute force attack**
- Try millions of passwords per second using GPUs

⚠️ Since normal hash functions (like SHA256) are **very fast**, this makes password cracking easier.

👉 **Solution:** Slow it down with **hash stretching**.

3 How Hash Stretching Works

◇ Basic idea:

Take a password and apply the hash function **many times** (e.g., 100,000 iterations).

Mathematically:

[
 $H_{\text{stretched}}(P) = H(H(H(\dots H(P + \text{salt})\dots)))$
]

Where:

- `H` = hash function (like SHA-256)
 - `P` = password
 - `salt` = random value added to make each password unique
 - The hash is applied **N times**
-

◇ Example (Python):

```
import hashlib

password = b"student123"
salt = b"xyz123"

# One round (normal hash)
h1 = hashlib.sha256(password + salt).hexdigest()

# Stretched hash (100,000 rounds)
h = password + salt
for i in range(100000):
    h = hashlib.sha256(h).digest()

print("Stretched hash:", h.hex())
```

This loop applies **SHA-256 100,000 times**, making it **100,000× slower** for both you and the attacker.

4 Visual Analogy

Think of **hash stretching** like a *password treadmill*:

- A normal hash = running 1 step 🏃
- Hash stretching = running 100,000 steps 🏃 🏃 🏃 🏃

💡 For the user, it's still just one login attempt.

🌟 For an attacker, it's 100,000 times more work per guess!

5 Real-World Implementations

Modern systems use **dedicated algorithms** that implement stretching *securely and efficiently*:

Algorithm	Based On	Features
PBKDF2 (Password-Based Key Derivation Function 2)	HMAC-SHA1/SHA256	Iteration count + salt
bcrypt	Blowfish cipher	Built-in salt, slow and adaptive
scrypt	PBKDF2 + memory-hard	Slows down GPU attacks
Argon2	(Modern standard)	Memory + time hardness

◇ Example: Using `hashlib.pbkdf2_hmac`

```
import hashlib, os

password = b"student123"
salt = os.urandom(16) # random 16-byte salt

key = hashlib.pbkdf2_hmac('sha256', password, salt, 100000)
print("Derived key:", key.hex())
```

Here:

- `'sha256'` → hash algorithm
- `salt` → random salt value
- `100000` → number of iterations (stretching factor)

Output:

```
Derived key: 4c20e58719a1f09b6edc68c7a6b908...
```

If an attacker tries to brute force, each guess costs **100,000 SHA-256 operations**.

6 Important Notes

Term	Meaning
Salt	Random data added before hashing to prevent rainbow-table attacks
Iterations	Number of times hashing is repeated
Stretching	Technique to slow down brute-force by increasing computational cost
Memory-hard functions	Advanced functions that use memory to slow down attackers with GPUs (e.g., scrypt, Argon2)

7 Comparison Example

Method	Iterations	Hashing Speed	Security
SHA-256	1	Very Fast	Weak
PBKDF2	100,000	Medium	Strong
bcrypt	Adaptive	Slow	Very Strong
Argon2	Adaptive	Slow + Memory-hard	Excellent

8 Summary / Key Takeaways

Concept	Explanation
Hash Stretching	Repeatedly hashing to slow down brute-force attacks
Why Needed	Because normal hashes are too fast
How Implemented	PBKDF2, bcrypt, scrypt, Argon2
Effect	Increases time per password guess
Salt	Ensures each password has a unique hash

